CO-2-ASSIGNMENT

Assignment title: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Course Outcome	CO2
Assignment	Assignment tool
Faculty	Dr. F. Mary Harin Fernandez
Student Name	Pooja Sree M
Register Number	192521403
Course Code	CSA0608
Course Name	Design and Analysis of Algorithm

Q1 — Sorting Under Different Data Conditions

Problem Description:

Sort e-commerce order priority scores using Selection Sort, Bubble Sort and Insertion Sort. The incoming data can be random, nearly sorted, sorted or highly disordered.

Algorithm Design: Selection Sort, Bubble Sort and Insertion Sort

Selection Sort — Logic

Divide the array into sorted and unsorted portions.

- Find the minimum element in the unsorted portion.
- Swap it with the first element of the unsorted portion.
- Repeat until all elements are sorted.
- Count comparisons and swaps to measure performance.

Bubble Sort — Logic

- Compare adjacent elements in the array.
- Swap them if they are in the wrong order.
- Continue each pass until the largest unsorted element reaches its correct position.
- Stop early if a complete pass performs no swaps.
- Record comparisons and swaps.

Insertion Sort — Logic

- Treat the first element as the sorted portion.
- Select the next element as the key.
- Shift larger elements to the right until the correct position is found.
- Insert the key into that position.
- Repeat until the array is sorted.
- Record comparisons and movements.

Design decision: Test all three algorithms using sorted, reverse-sorted, and duplicate-containing datasets. Compare their operation counts and recommend Insertion Sort for maintaining the order when approximately 95% of the orders are already sorted and small batches are added.

Predictions before testing :

Algorithm	Prediction	Reason
Selection Sort	Similar comparison count for every arrangement: $n(n-1)/2$.	It searches the remaining suffix for the minimum on every pass.
Bubble Sort (early stop)	Fast on sorted/nearly sorted inputs; quadratic on reverse/random inputs.	It stops after a pass with no swaps, but may perform many adjacent swaps.
Insertion Sort	Very fast on nearly sorted inputs; quadratic on reverse/random inputs.	Work is proportional to the number of inversions; it shifts elements already larger than the key.

Designed test datasets:

Condition	Example dataset	Purpose
Contrasting A: already sorted	[1, 2, 3, 4, 5, 6]	Tests early termination and best-case behaviour.
Contrasting B: reverse sorted	[6, 5, 4, 3, 2, 1]	Creates many inversions and exposes shifts/swaps.
Additional edge case: duplicates/empty/singleton	[4, 2, 2, 4, 1]; also [] and [7]	Checks duplicates and boundary sizes.

Pseudocode:

```
SELECTION-SORT(A)
  for i = 0 to n-2
    minIndex = i
    for j = i+1 to n-1
      comparisons += 1
      if A[j] < A[minIndex]: minIndex = j
    if minIndex != i: swap A[i], A[minIndex]; swaps += 1

BUBBLE-SORT(A)
  for end = n-1 down to 1
    swapped = false
    for j = 0 to end-1
      comparisons += 1
      if A[j] > A[j+1]: swap; swaps += 1; swapped = true
    if swapped == false: break

INSERTION-SORT(A)
  for i = 1 to n-1
    key = A[i]; j = i-1
    while j >= 0
      comparisons += 1
      if A[j] > key: A[j+1] = A[j]; moves += 1; j -= 1
      else: break
    A[j+1] = key; moves += 1
```

Complexity analysis:

Algorithm	Best case	Average case	Worst case	Auxiliary space
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Bubble Sort (early stop)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$

Experimental evidence: random input comparisons

n	Selection comparisons	Bubble comparisons	Insertion comparisons
10	45	45	29
50	1225	1170	644
100	4950	4884	2813
200	19900	19522	9244

Operation counts on designed conditions (n = 6)

Dataset	Algorithm	Key comparisons	Swaps/moves (implementation-specific)
Sorted	Selection	15	0
Sorted	Bubble	5	0
Sorted	Insertion	5	5
Reverse	Selection	15	3
Reverse	Bubble	15	15
Reverse	Insertion	15	20
Duplicates	Selection	15	3
Duplicates	Bubble	15	10
Dataset	Algorithm	Key comparisons	Swaps/moves (implementation-specific)

Source code :

```
import random
import turtle
def selection_sort(a):
    c = 0
    for i in range(len(a) - 1):
        m = i
        for j in range(i + 1, len(a)):
            c += 1
```

```
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
    return c
def bubble_sort(a):
    c = 0
    for end in range(len(a) - 1, 0, -1):
        changed = False
        for j in range(end):
            c += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                changed = True
        if not changed:
            break
    return c
def insertion_sort(a):
    c = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            c += 1
            if a[j] > key:
                a[j + 1] = a[j]
                j -= 1
            else:
```

```
            break
        a[j + 1] = key
    return c
sizes = [10, 50, 100, 200]
sel, bub, ins = [], [], []
random.seed(42)
for n in sizes:
    data = [random.randint(1, 10000) for _ in range(n)]
    sel.append(selection_sort(data.copy()))
    bub.append(bubble_sort(data.copy()))
    ins.append(insertion_sort(data.copy()))
print("Input sizes:", sizes)
print("Selection:", sel)
print("Bubble:", bub)
print("Insertion:", ins)

# Draw graph using turtle
screen = turtle.Screen()
screen.title("Sorting Algorithm Comparisons")
screen.setup(1000, 650)
screen.bgcolor("white")
pen = turtle.Turtle()
pen.speed(0)
pen.hideturtle()
pen.width(3)
left, bottom = -400, -230
graph_width, graph_height = 800, 430
```

```
max_value = max(sel + bub + ins) * 1.1

# Draw axes
pen.penup()
pen.goto(left, bottom)
pen.pendown()
pen.goto(left, bottom + graph_height)
pen.goto(left, bottom)
pen.goto(left + graph_width, bottom)

# Axis labels
pen.penup()
pen.goto(0, -290)
pen.write("Input size n", align="center", font=("Arial", 14, "normal"))

pen.goto(-450, 230)
pen.write("Comparisons", align="center", font=("Arial", 12, "normal"))

# X-axis labels
for n in sizes:
    x = left + (n / 200) * graph_width
    pen.penup()
    pen.goto(x, bottom - 25)
    pen.write(str(n), align="center", font=("Arial", 10, "normal"))

# Y-axis labels and grid
for k in range(5):
```

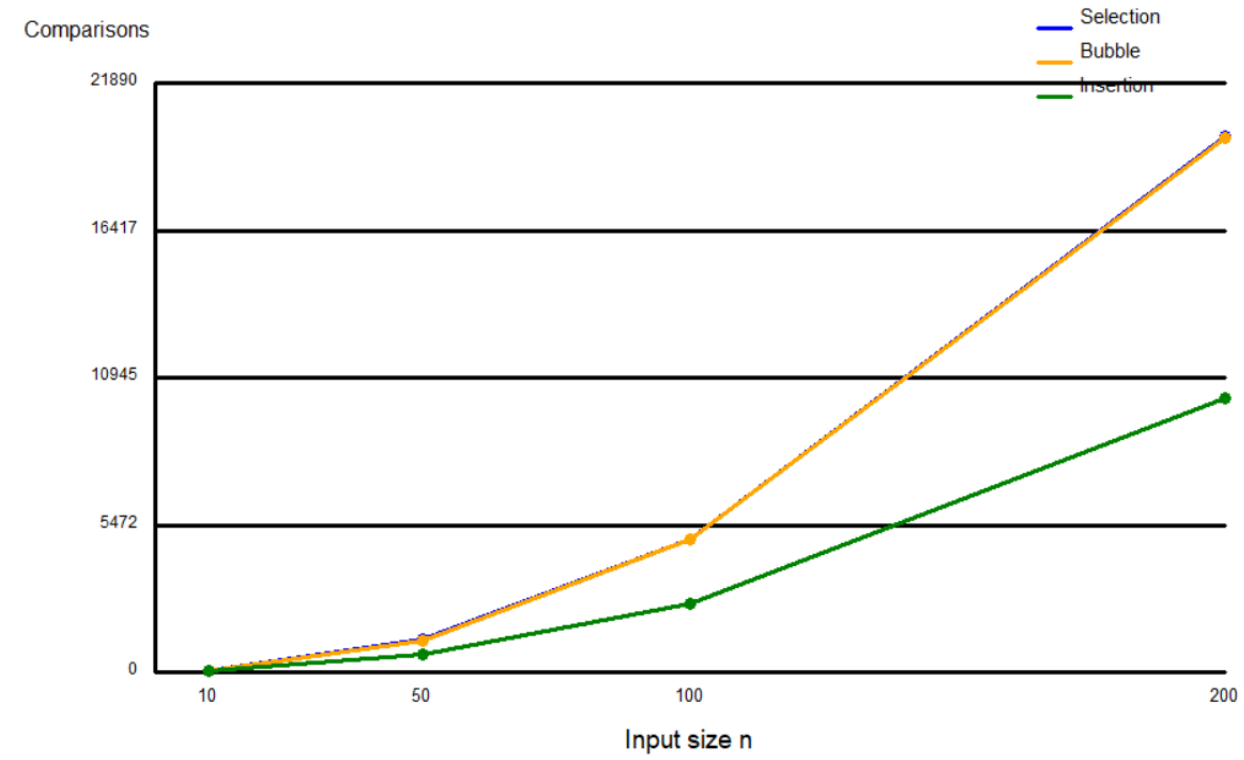
```
    value = max_value * k / 4
    y = bottom + graph_height * k / 4
    pen.penup()
    pen.goto(left - 12, y - 5)
    pen.write(str(int(value)), align="right", font=("Arial", 9, "normal"))
    pen.goto(left, y)
    pen.pendown()
    pen.goto(left + graph_width, y)
    pen.penup()


def draw_line(values, color, name):
    pen.color(color)
    pen.width(3)
    for i, n in enumerate(sizes):
        x = left + (n / 200) * graph_width
        y = bottom + (values[i] / max_value) * graph_height
        if i == 0:
            pen.penup()
            pen.goto(x, y)
            pen.pendown()
        else:
            pen.goto(x, y)
        pen.dot(9)
```

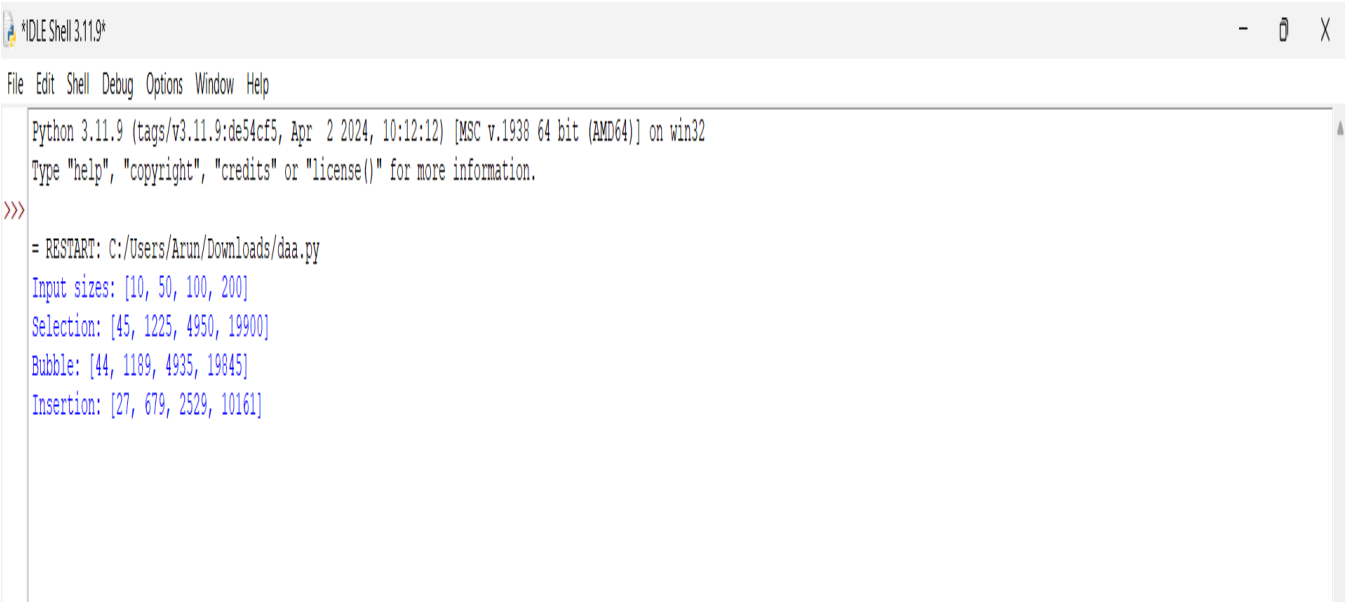
```
# Draw three sorting lines
draw_line(sel, "blue", "Selection")
draw_line(bub, "orange", "Bubble")
draw_line(ins, "green", "Insertion")
pen.penup()
pen.goto(0, 270)
pen.color("black")
pen.write("Q1: Comparisons on random data",
          align="center", font=("Arial", 18, "normal"))
for i, (color, name) in enumerate([
    ("blue", "Selection"),
    ("orange", "Bubble"),
    ("green", "Insertion")
]):
    pen.goto(260, 240 - i * 25)
    pen.color(color)
    pen.pendown()

    pen.forward(25)
    pen.penup()
    pen.forward(8)
    pen.color("black")
    pen.write(name, font=("Arial", 11, "normal"))
turtle.done()
```

GRAPH:



OUTPUT:



Observation and design decision:

Selection Sort's comparisons are insensitive to input order, although its swaps are limited. Bubble Sort with early stopping is useful for already-sorted data but can make many swaps on reverse order. Insertion Sort benefits most from small batches inserted into an already ordered list. Because approximately 95% of orders are already correctly ordered and only small batches arrive periodically, recommend Insertion Sort for maintaining the order of the small incoming batch (or inserting each new order into its correct position). If the full dataset must be repeatedly re-sorted, use measured workload and data size to reassess. This recommendation follows the stated data condition rather than claiming one algorithm is always best.

Q2 — When Is Sequential Search Appropriate?

Problem Description:

A frequently changing list of aircraft maintenance tools, with each record containing Tool ID, Technician ID, Issue Time and Work Bay.

Algorithm Design: Searching Aircraft Maintenance Tools

1. Accept the list of tool records and the target Tool ID.
2. Initialize the comparison counter to zero.
3. Start searching from the first record.
4. Compare each record's Tool ID with the target.
5. Increment the comparison counter after each comparison.
6. If a match is found, return the corresponding technician and tool details.
7. If the target is not found, continue until the list ends.
8. If the entire list is checked, return NOT FOUND.
9. Repeat the experiment with successful searches at different positions, unsuccessful searches, and different list sizes.

10. Use the comparison counts to decide when sequential search becomes inefficient.

Design decision: Sequential search is appropriate for small, frequently changing lists. For larger lists and more frequent requests across 20 facilities, consider an indexed search structure or database index.

Pseudocode:

```
SEQUENTIAL-SEARCH(records, targetToolID)
  comparisons = 0
  for i = 0 to length(records)-1
    comparisons += 1
    if records[i].ToolID == targetToolID
      return (records[i], comparisons)
  return (NOT_FOUND, comparisons)
```

Test dataset and experiments:

Use Tool IDs T-101 through T-110 as a simple ordered example. Successful searches at the beginning, middle and end take 1, 5 and 10 comparisons respectively; a missing ID takes 10 comparisons.

List size n	Successful: first	Successful: middle	Successful: last	Unsuccessful
10	1	5	10	10
100	1	50	100	100
1000	1	500	1000	1000

Complexity and mathematical analysis:

Best case: 1 comparison = $\Theta(1)$. Average successful search in a uniformly ordered list with equally likely positions: $(n+1)/2$ comparisons = $\Theta(n)$. Worst case, including a missing target: n comparisons = $\Theta(n)$. Auxiliary space: $\Theta(1)$.

Source code :

```
import turtle
def sequential_search(records, target):
    comparisons = 0
    for i, record in enumerate(records):
        comparisons += 1
        if record['tool_id'] == target:
            return record, comparisons
    return None, comparisons

if __name__ == '__main__':
    tools = [
        {'tool_id': f'T-{i:03}',
         'technician': f'TECH-{i%4+1}',
         'issue_time': '10:00',
         'work_bay': f'BAY-{i%3+1}'}
        for i in range(101, 111)
    ]

    targets = ['T-101', 'T-105', 'T-110', 'T-999']
    counts = []

    for target in targets:
        record, count = sequential_search(tools, target)
        print(target, 'FOUND' if record else 'NOT_FOUND',
              'comparisons =', count)
        counts.append(count)

    # Create graph window
    screen = turtle.Screen()
    screen.title("Sequential Search Performance")
    screen.setup(900, 600)
    screen.bgcolor("white")
    pen = turtle.Turtle()
    pen.hideturtle()
```

```
pen.speed(0)
pen.width(3)

# Graph dimensions
left = -320
bottom = -180
width = 640
height = 300
max_count = 10

# Draw axes
pen.penup()
pen.goto(left, bottom)
pen.pendown()
pen.goto(left, bottom + height)
pen.goto(left, bottom)
pen.goto(left + width, bottom)

# Graph title
pen.penup()
pen.goto(0, 230)
pen.write("Sequential Search: Key Comparisons",
        align="center", font=("Arial", 16, "bold"))

# Axis labels
pen.goto(0, -240)
pen.write("Search Target", align="center",
        font=("Arial", 12, "normal"))
pen.goto(-400, 150)
pen.write("Comparisons", align="center",
        font=("Arial", 11, "normal"))

# Y-axis scale and grid
for value in range(0, max_count + 1, 2):
    y = bottom + (value / max_count) * height
```

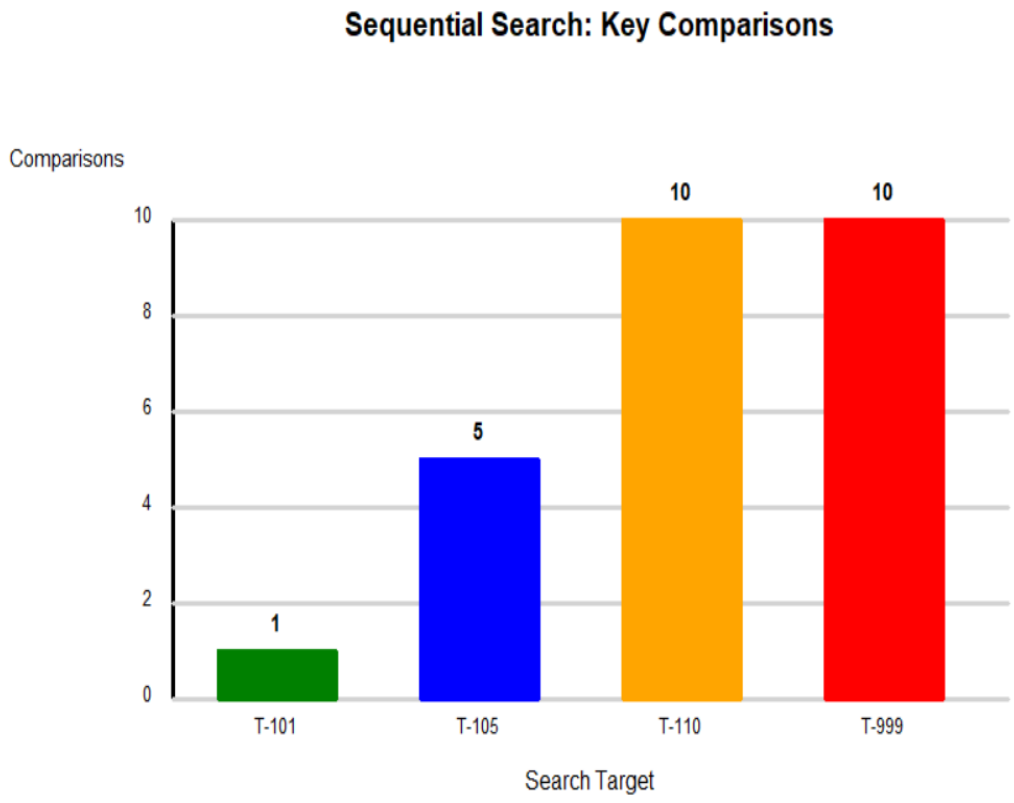
```
        pen.penup()
        pen.goto(left - 15, y - 5)
        pen.write(str(value), align="right",
                  font=("Arial", 10, "normal"))
        pen.goto(left, y)
        pen.color("lightgray")
        pen.pendown()
        pen.goto(left + width, y)
        pen.penup()
        pen.color("black")

    # Draw bars
    colors = ["green", "blue", "orange", "red"]
    bar_width = 90
    gap = 65
    for i, count in enumerate(counts):
        x = left + 35 + i * (bar_width + gap)
        bar_height = (count / max_count) * height
        pen.penup()
        pen.goto(x, bottom)
        pen.color(colors[i])
        pen.pendown()
        pen.begin_fill()
        pen.goto(x + bar_width, bottom)
        pen.goto(x + bar_width, bottom + bar_height)
        pen.goto(x, bottom + bar_height)
        pen.goto(x, bottom)
        pen.end_fill()

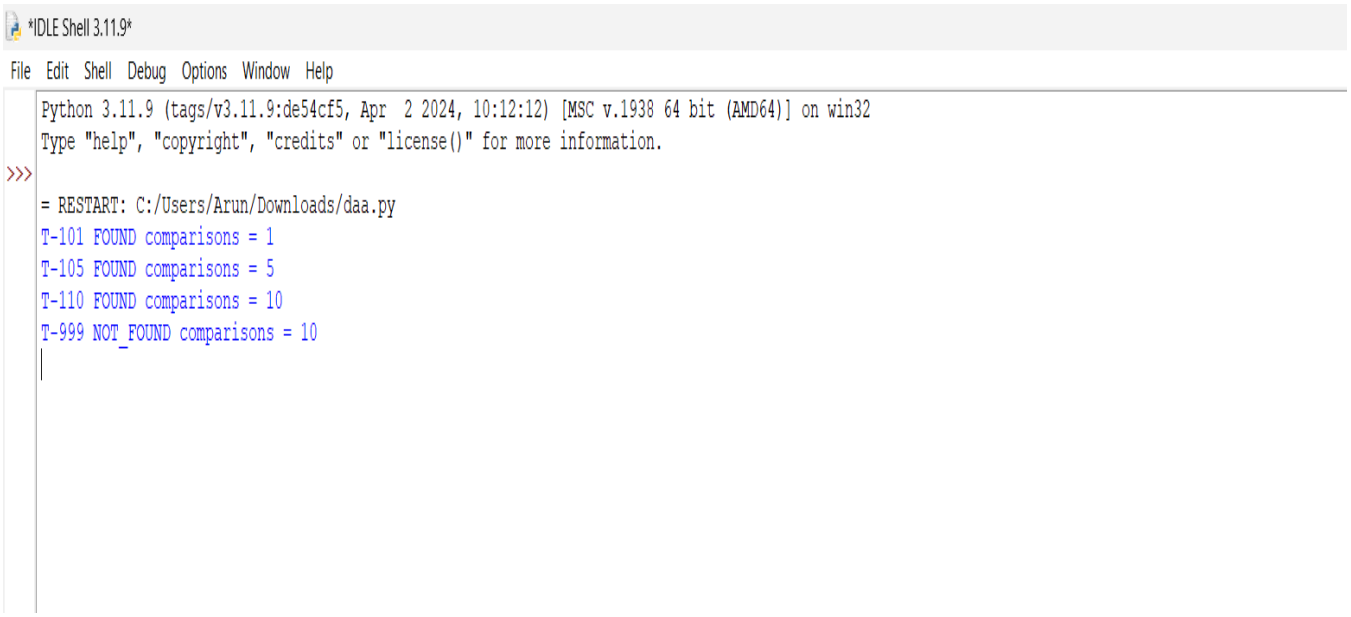
# Show comparison count
        pen.penup()
        pen.goto(x + bar_width / 2,
                 bottom + bar_height + 8)
        pen.color("black")
```

```
        pen.write(str(count), align="center",
                  font=("Arial", 11, "bold"))
        # Target labels
        pen.goto(x + bar_width / 2, bottom - 25)
        pen.write(targets[i], align="center",
                  font=("Arial", 10, "normal"))
    turtle.done()
```

GRAPH:



OUTPUT:



Engineering decision: sequential search is reasonable for small, short-lived lists, infrequent lookups, or when records are not sorted and building an index would cost more than searching. Reconsider it when lists become large or lookup frequency grows. With 20 facilities and many active records/requests, use an indexed lookup structure (for example, a hash table keyed by Tool ID) or a database index, while handling updates and consistency.

Q3 — Brute-Force String Matching

Problem Description:

Search security logs for a known event signature. The algorithm tries every possible alignment of a pattern in the text and compares characters from left to right.

Algorithm Design: Searching Security Logs for Event Signatures

1. Accept the log text and the pattern to be searched.
2. Calculate the lengths of the text and pattern.
3. Align the pattern with the beginning of the text.
4. Compare pattern characters with corresponding text characters from left to right.
5. Count every character comparison.

6. If a mismatch occurs, shift the pattern one position to the right.
7. If every pattern character matches, return the starting position.
8. Continue until all possible alignments have been examined.
9. If no match is found, return NOT FOUND.
10. Test three conditions: an early match, repeated-prefix comparisons, and an unsuccessful search.
11. Compare the predicted and actual character comparison counts.

Design decision: The algorithm is simple but may repeat many character comparisons when the text and pattern contain repeated characters.

Pseudocode:

```
BRUTE-FORCE-MATCH(text, pattern)
  comparisons = 0
  for shift = 0 to n-m
    j = 0
    while j < m
      comparisons += 1
      if text[shift+j] != pattern[j]: break
      j += 1
    if j == m: return (shift, comparisons)
  return (NOT_FOUND, comparisons)
```

Three designed test conditions and predictions:

Case	Text	Pattern	Prediction
Low work / early match	security-alert	sec	Match at shift 0 after 3 equal comparisons.
High repeated work	AAAAAAAAAAAB	AAAAB	Many alignments share four matching A characters before mismatch; then match near end.

Reproducible comparison counts:

Case	Text	Pattern	Result index	Character comparisons
Early match	security-alert	sec	0	3
Repeated prefix	AAAAAAAAAAAB	AAAAB	7	40
Not found	AAAAAAAAAA	AAAB	-1	28

Complexity analysis:

Let n be text length and m be pattern length. Worst-case comparisons are at most $(n-m+1)m$, so time is $O((n-m+1)m)$, commonly written $O(nm)$; best case can be $\Theta(m)$ when a match begins at the first position. Auxiliary space is $\Theta(1)$. Repeated prefixes in the pattern/text cause the same characters to be compared again at nearby shifts.

Source code :

```
import turtle
def brute_force_match(text, pattern):
    comparisons = 0
```

```
    for shift in range(len(text) - len(pattern) + 1):
        j = 0
        while j < len(pattern):
            comparisons += 1
            if text[shift + j] != pattern[j]:
                break
            j += 1
        if j == len(pattern):
            return shift, comparisons
    return -1, comparisons

# Test cases
tests = [
    ('security-alert', 'sec'),
    ('AAAAAAAAAAAB', 'AAAAB'),
    ('AAAAAAAAAA', 'AAAB')
]

counts = []
for text, pattern in tests:
    index, count = brute_force_match(text, pattern)
    print(repr(text), repr(pattern),
          'index =', index, 'comparisons =', count)
    counts.append(count)

# Draw graph using turtle
screen = turtle.Screen()
screen.title("Brute Force String Matching")
screen.setup(900, 600)
screen.bgcolor("white")

pen = turtle.Turtle()
pen.hideturtle()
pen.speed(0)
pen.width(2)
```

```
left = -320
bottom = -180
graph_width = 640
graph_height = 300
max_count = max(counts) + 2

# Draw axes
pen.penup()
pen.goto(left, bottom)
pen.pendown()
pen.goto(left, bottom + graph_height)
pen.goto(left, bottom)
pen.goto(left + graph_width, bottom)

# Title
pen.penup()
pen.goto(0, 230)
pen.write("Brute Force String Matching",
        align="center", font=("Arial", 16, "bold"))

# Axis labels
pen.goto(0, -245)
pen.write("Test Cases", align="center",
        font=("Arial", 12, "normal"))
pen.goto(-400, 150)
pen.write("Comparisons", align="center",
        font=("Arial", 11, "normal"))

# Y-axis markings and grid
for value in range(0, max_count + 1, 2):
    y = bottom + value / max_count * graph_height
    pen.penup()
    pen.goto(left - 15, y - 5)
    pen.write(str(value), align="right",
```

```
            font=("Arial", 10, "normal"))
    pen.goto(left, y)
    pen.color("lightgray")
    pen.pendown()
    pen.goto(left + graph_width, y)
    pen.penup()
    pen.color("black")

# Draw bars
labels = ["Test 1", "Test 2", "Test 3"]
colors = ["blue", "orange", "green"]
bar_width = 100
gap = 105

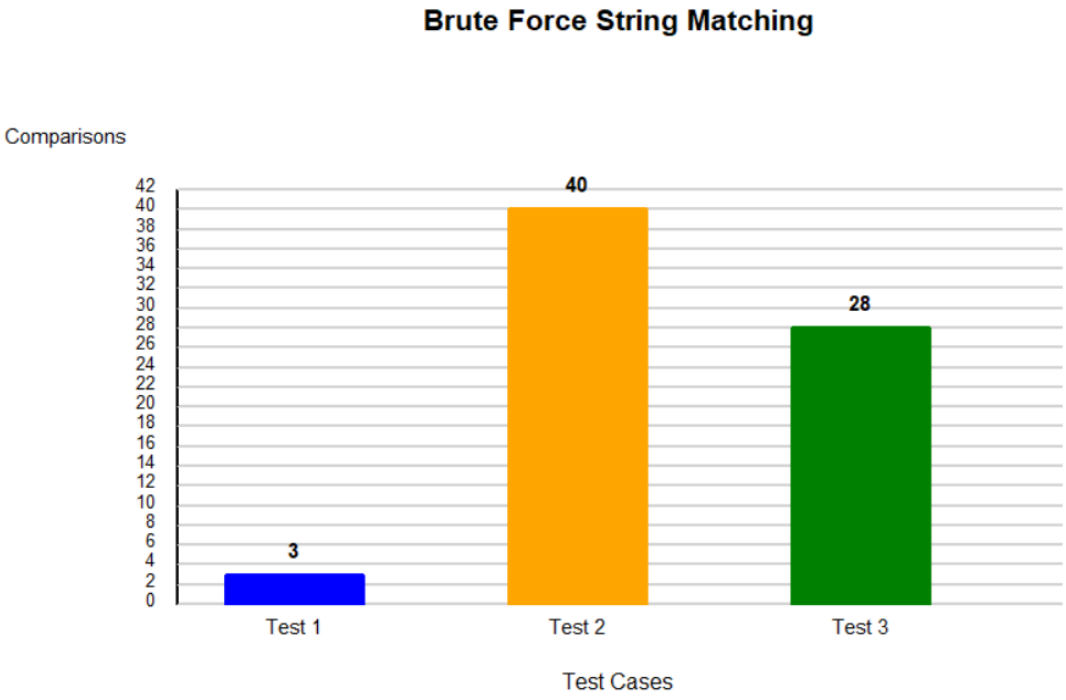
for i, count in enumerate(counts):
    x = left + 35 + i * (bar_width + gap)
    bar_height = count / max_count * graph_height
    pen.penup()
    pen.goto(x, bottom)
    pen.color(colors[i])
    pen.pendown()
    pen.begin_fill()
    pen.goto(x + bar_width, bottom)
    pen.goto(x + bar_width, bottom + bar_height)
    pen.goto(x, bottom + bar_height)
    pen.goto(x, bottom)
    pen.end_fill()

    pen.penup()
    pen.color("black")
    pen.goto(x + bar_width / 2, bottom + bar_height + 8)
    pen.write(str(count), align="center",
            font=("Arial", 11, "bold"))
    pen.goto(x + bar_width / 2, bottom - 25)
    pen.write(labels[i], align="center",
```

```
        font=("Arial", 11, "normal"))

turtle.done()
```

GRAPH:



OUTPUT:

```
*IDLE Shell 3.11.9*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Arun/Downloads/daa.py
'security-alert' 'sec' index = 0 comparisons = 3
'AAAAAAAAAAB' 'AAAAB' index = 7 comparisons = 40
'AAAAAAAAAA' 'AAAB' index = -1 comparisons = 28
```

The experiment shows that input length alone is not enough to predict exact work. Pattern length, repeated prefixes, mismatch position and match position affect the number of character comparisons.

Q4 — Brute-Force Closest Pair

Problem Description:

Given n drone coordinates (xi, yi), examine every unique pair and keep the pair with minimum Euclidean distance.

Algorithm Design: Finding the Two Closest Drones

1. Accept the coordinates of all drones.
2. Initialize the minimum distance to infinity.
3. Generate every unique pair of drones.
4. Avoid duplicate comparisons by considering only pairs where the second index is greater than the first.
5. Calculate the Euclidean distance for each pair:

   $d = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$
6. Count each pair examined and each distance calculation.
7. Compare the calculated distance with the current minimum distance.
8. If it is smaller, update the minimum distance and closest pair.
9. Repeat until all unique pairs have been checked.
10. Display the closest pair, minimum distance, and total calculations.
11. Test at least three input sizes and compare experimental pair counts with the theoretical formula:

    $n(n-1)/2$

Design decision: Brute-force closest pair is suitable for small datasets but becomes inefficient for large datasets because the number of pairs grows quadratically.

Pair-count derivation:

For each i, compare only with j > i. This avoids checking both (i,j) and (j,i), and avoids comparing a point with itself. The number of pairs is $(n-1)+(n-2)+\ldots+1 = n(n-1)/2 = C(n,2)$. Each pair requires one distance calculation.

Pseudocode:

```
CLOSEST-PAIR(points)
  bestDistance = infinity; bestPair = NONE; pairCount = 0
  for i = 0 to n-2
    for j = i+1 to n-1
      pairCount += 1
      d = sqrt((x[i]-x[j])^2 + (y[i]-y[j])^2)
      if d < bestDistance: bestDistance = d; bestPair = (i,j)
  return bestPair, bestDistance, pairCount
```

Experimental growth:

n points	Theoretical unique pairs	Distance calculations
10	45	45
100	4950	4950
1000	499500	499500

The count grows quadratically: doubling n approximately quadruples pair checks. For a response-time criterion, define 1 second as the target for a single closest-pair query in the chosen deployment environment; benchmark on that hardware to estimate the largest feasible n. This report does not invent a machine-specific runtime threshold.

Complexity analysis:

Time: $\Theta(n^2)$, because all unique pairs are examined. Auxiliary space: $\Theta(1)$, excluding the input. The bottleneck is exhaustive pair enumeration, not the distance formula itself. A redesigned method should sort points by a coordinate and eliminate most pairs using geometric bounds, rather than calculating every distance.

Source code :

```
from math import hypot, inf
import turtle
def closest_pair(points):
    best_distance = inf
    best_pair = None
    pair_count = 0
    for i in range(len(points) - 1):
        for j in range(i + 1, len(points)):
            pair_count += 1
            d = hypot(points[i][0] - points[j][0],
                      points[i][1] - points[j][1])
            if d < best_distance:
                best_distance = d
                best_pair = (points[i], points[j])
    return best_pair, best_distance, pair_count

points = [(0, 0), (5, 5), (2, 2), (9, 1), (2, 3)]
pair, distance, count = closest_pair(points)
print("Closest pair:", pair)
print("Minimum distance:", round(distance, 2))
print("Pairs checked:", count)

# Draw graph using turtle
screen = turtle.Screen()
screen.title("Closest Pair of Points")
screen.setup(800, 650)
screen.bgcolor("white")
pen = turtle.Turtle()
pen.speed(0)
pen.hideturtle()

# Coordinate mapping
def screen_point(p):
    return (p[0] * 45 - 200, p[1] * 45 - 150)
```

```
# Draw coordinate axes
pen.penup()
pen.goto(-250, -170)
pen.pendown()
pen.goto(250, -170)
pen.penup()
pen.goto(-250, -170)
pen.pendown()
pen.goto(-250, 230)

# Axis labels and title
pen.penup()
pen.goto(0, 250)
pen.write("Closest Pair of Points",
        align="center", font=("Arial", 16, "bold"))
pen.goto(0, -210)
pen.write("X-axis", align="center", font=("Arial", 11, "normal"))
pen.goto(-300, 220)
pen.write("Y-axis", align="center", font=("Arial", 11, "normal"))

# Draw all points
for p in points:
    x, y = screen_point(p)
    pen.penup()
    pen.goto(x, y)
    pen.dot(10, "blue")
    pen.goto(x + 8, y + 8)
    pen.write(str(p), font=("Arial", 10, "normal"))

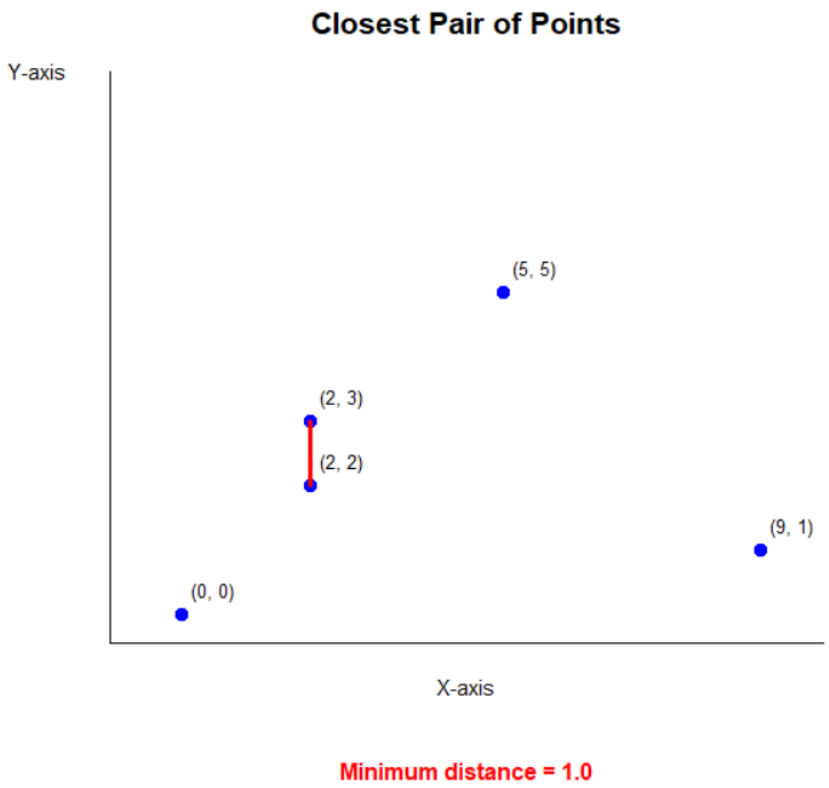
# Highlight closest pair
pen.color("red")
pen.width(3)
p1, p2 = pair
pen.penup()
```

```
pen.goto(screen_point(p1))
pen.pendown()
pen.goto(screen_point(p2))

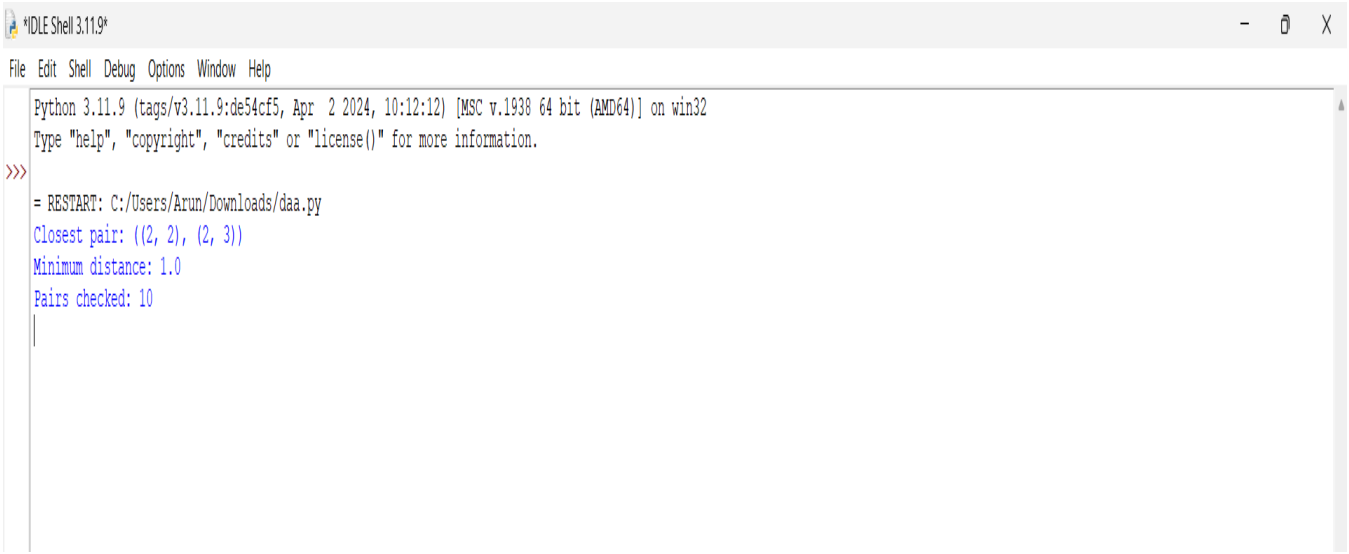
# Draw information
pen.penup()
pen.goto(0, -270)
pen.color("red")
pen.write("Minimum distance = " + str(round(distance, 2)),
        align="center", font=("Arial", 12, "bold"))

turtle.done()
```

GRAPH:



OUTPUT:



Q5 — Brute-Force Convex Hull

Problem Description:

Monitoring stations are points in a plane. The convex hull is the smallest convex boundary containing all stations.

Algorithm Design: Finding the Outer Boundary of Monitoring Stations

1. Accept the coordinates of all monitoring stations.
2. Select every possible pair of distinct points as a candidate boundary edge.
3. For each pair A and B, consider the line joining them.
4. Examine every remaining point P.
5. Calculate the orientation using the cross-product expression:

   C = (x_B - x_A)(y_P - y_A) - (y_B - y_A)(x_P - x_A)
6. Determine whether each point lies to the positive side, negative side, or directly on the line.
7. If points exist on both sides of the line, reject the candidate edge.
8. Otherwise, retain the pair as a supporting-line candidate.

9. Handle collinear points and remove duplicate candidate edges.
10. Arrange the accepted boundary edges in order to construct the convex hull.
11. Visualize the hull and test datasets containing interior points and challenging configurations.
12. Analyze how the number of orientation tests grows with the number of points.

Design decision: The supporting-line test identifies potential hull edges because a genuine convex boundary edge has all points on one side of its supporting line. Collinear points require careful handling when constructing the final boundary.

Geometric decision rule:

For candidate points A and B, compute the signed cross product for every other point P: cross=(Bx−Ax)(Py−Ay)−(By−Ay)(Px−Ax). Positive and negative signs mean points occur on opposite sides of line AB. If points occur on both sides, AB is not a hull edge. If all points lie on one side or on the line, AB is a supporting-line candidate.

Pseudocode:

```
BRUTE-FORCE-HULL(points)
  edges = empty set
  for each unordered pair A, B
    hasPositive = false; hasNegative = false
    for each point P other than A and B
      z = cross(A, B, P)
      if z > 0: hasPositive = true
      if z < 0: hasNegative = true
    if not (hasPositive and hasNegative):
      add candidate edge AB
  construct and order the boundary edges; handle collinear points consistently
  return hull boundary
```

Test configurations:

Test	Point set	Purpose
Interior points	[(0,0),(4,0),(4,4),(0,4),(2,2),(1,2)]	Hull should be the four outer

		corners; interior points are excluded.
Challenge: collinear edge points	[(0,0),(2,0),(4,0),(4,3),(0,3),(2,1)]	Checks collinearity and whether points on a hull edge are handled consistently.

Implementation note: the brute-force supporting-line test identifies candidate boundary lines/edges; collinear points can cause multiple point pairs to lie on the same boundary line. A production implementation must deduplicate/order edges and decide whether to retain or remove intermediate collinear boundary points. The visualization below uses a consistently ordered convex boundary for display.

Complexity and correctness:

There are $\Theta(n^2)$ candidate point pairs, and each candidate checks up to $n-2$ other points, giving $O(n^3)$ time and $O(n^2)$ worst-case candidate-edge storage. Correctness intuition: a line segment between hull vertices is a supporting edge when every point is on one side of its supporting line. If points lie on both sides, the segment passes through the interior and cannot be a hull boundary edge.

Source code :

```
import turtle


def cross(a, b, p):
    return ((b[0] - a[0]) * (p[1] - a[1])
            - (b[1] - a[1]) * (p[0] - a[0]))


def brute_force_hull_edges(points):
    edges = []
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            positive = negative = False
            for k in range(n):
                if k == i or k == j:
                    continue
```

```
            z = cross(points[i], points[j], points[k])
            if z > 0:
                positive = True
            elif z < 0:
                negative = True
            if positive and negative:
                break
        if not (positive and negative):
            edges.append((points[i], points[j]))

    return edges

pts = [(0, 0), (4, 0), (4, 4),
       (0, 4), (2, 2), (1, 2)]
edges = brute_force_hull_edges(pts)
print("Supporting-line candidate edges:")
for edge in edges:
    print(edge)

# Draw graph using turtle
screen = turtle.Screen()
screen.title("Brute Force Convex Hull")
screen.setup(800, 650)
screen.bgcolor("white")
pen = turtle.Turtle()
pen.speed(0)
pen.hideturtle()
pen.width(3)

def convert(p):
    return (p[0] * 80 - 160, p[1] * 80 - 120)

# Draw coordinate axes
pen.penup()
pen.color("gray")
```

```
pen.goto(-200, -140)
pen.pendown()
pen.goto(240, -140)
pen.penup()
pen.goto(-200, -140)
pen.pendown()
pen.goto(-200, 240)

# Draw hull boundary edges
pen.color("red")
pen.width(4)
for a, b in edges:
    pen.penup()
    pen.goto(convert(a))
    pen.pendown()
    pen.goto(convert(b))

# Draw and label all points
for p in pts:
    x, y = convert(p)
    pen.penup()
    pen.goto(x, y)
    pen.dot(12, "blue")
    pen.goto(x + 10, y + 8)
    pen.color("black")
    pen.write(str(p), font=("Arial", 11, "normal"))

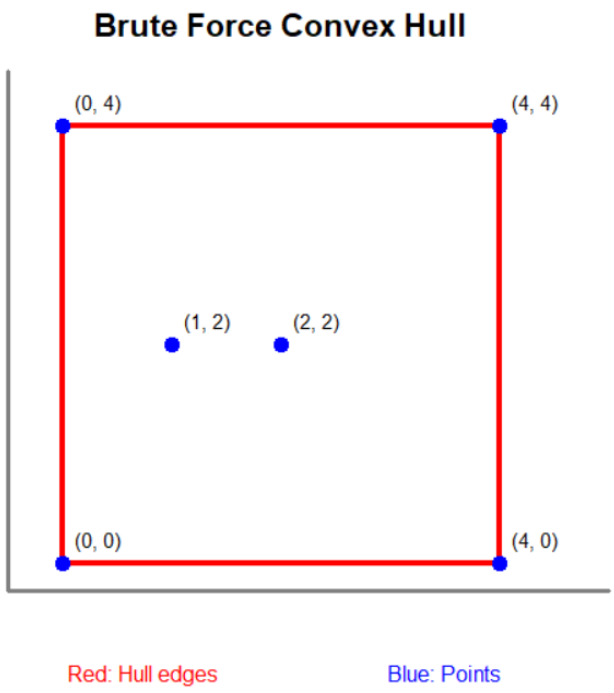
# Title and legend
pen.penup()
pen.goto(0, 260)
pen.write("Brute Force Convex Hull",
          align="center", font=("Arial", 17, "bold"))

pen.goto(-100, -210)
pen.color("red")
```

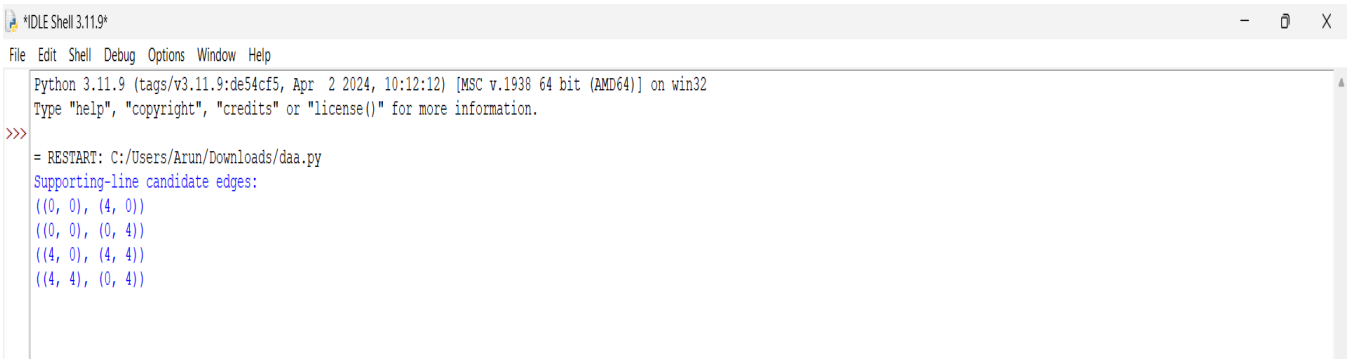
```
pen.write("Red: Hull edges",
        align="center", font=("Arial", 12, "normal"))

pen.goto(120, -210)
pen.color("blue")
pen.write("Blue: Points",
        align="center", font=("Arial", 12, "normal"))
turtle.done()
```

GRAPH:



OUTPUT:



Q6 — Exhaustive Search and Combinatorial Growth

Problem Description:

Choose packages for a humanitarian airlift. Each package has a weight and humanitarian value. Total weight must not exceed payload capacity, and total value must be maximized.

Algorithm Design: Humanitarian Airlift Package Selection

1. Accept the available packages, their weights, their humanitarian values, and the payload capacity.
2. Represent each possible package combination as a subset, using a binary choice for each package: selected or not selected.
3. Generate every possible subset, including the empty subset.
4. Calculate the total weight and total value of each subset.
5. Check whether the total weight exceeds the payload capacity.
6. Reject infeasible subsets.
7. Compare the value of each feasible subset with the best value found so far.
8. If a better value is found, update the best value and selected subset.
9. Continue until all subsets have been examined.
10. Return the package combination with the highest feasible humanitarian value.
11. Verify the result using a small dataset where all combinations can be checked independently.
12. Test at least three problem sizes and compare the number of subsets examined with \(2^n\).

Design decision: Exhaustive search guarantees the optimum when all subsets are examined, but its exponential growth makes it unsuitable for sufficiently large numbers of packages.

Pseudocode:

```
EXHAUSTIVE-PACKING(packages, capacity)
  bestValue = -infinity; bestSubset = empty
  for mask = 0 to 2^n - 1
    weight = 0; value = 0; subset = empty
    for i = 0 to n-1
```

```
    if bit i of mask is 1
      add package i to subset
      weight += packages[i].weight
      value += packages[i].value
  if weight <= capacity and value > bestValue
    bestValue = value; bestSubset = subset
return bestSubset, bestValue
```

Small correctness-check instance :

Package	Weight	Value
P1	2	6
P2	3	8
P3	4	9
P4	5	10

Capacity = 7. Enumerate all $2^4 = 16$ subsets, including the empty subset. The best feasible value is 17 from P1 + P3 (weight 6, value 15) or P2 + P3 (weight 7, value 17); therefore the optimum is P2 + P3, total weight 7 and total value 17.

Growth derivation and experiment:

Each of n packages is either selected or not selected, giving 2 choices per package and 2^n subsets overall. Increasing n by one doubles the number of subsets. Every subset is evaluated, so the time is $O(n \cdot 2^n)$ for this implementation, with O(n) auxiliary working space.

n packages	Subsets examined (2^n)	Subset-item checks upper bound ($n \cdot 2^n$)
4	16	64
10	1024	10240
15	32768	491520
20	1048576	20971520

Source code :

```
import turtle
def exhaustive_packing(packages, capacity):
    n = len(packages)
    best_value = -1
    best_weight = 0
    best_mask = 0
    for mask in range(1 << n):
        weight = value = 0
        for i, (w, v) in enumerate(packages):
            if mask & (1 << i):
                weight += w
                value += v
        if weight <= capacity and value > best_value:
            best_value = value
            best_weight = weight
            best_mask = mask

    chosen = [i for i in range(n)
              if best_mask & (1 << i)]

    return chosen, best_weight, best_value

packages = [(2, 6), (3, 8), (4, 9), (5, 10)]
```

```
capacity = 7
chosen, weight, value = exhaustive_packing(packages, capacity)
print("Chosen package indices:", chosen)
print("Total weight:", weight)
print("Total value:", value)
print("Capacity:", capacity)

# Draw graph using turtle
screen = turtle.Screen()
screen.title("Exhaustive Packing - Knapsack")
screen.setup(850, 600)
screen.bgcolor("white")

pen = turtle.Turtle()
pen.speed(0)
pen.hideturtle()

# Graph settings
left = -300
bottom = -170
bar_width = 100
gap = 65
scale = 25

# Title
pen.penup()
pen.goto(0, 230)
pen.write("0/1 Knapsack: Package Values",
        align="center", font=("Arial", 17, "bold"))

# Axes
pen.goto(left, bottom)
pen.pendown()
pen.goto(left, bottom + 300)
pen.goto(left, bottom)
```

```
pen.goto(300, bottom)
pen.penup()

# Axis labels
pen.goto(0, -220)
pen.write("Packages", align="center",
        font=("Arial", 12, "normal"))
pen.goto(-370, 170)
pen.write("Value", align="center",
        font=("Arial", 12, "normal"))

# Draw bars for each package
for i, (w, v) in enumerate(packages):
    x = left + 35 + i * (bar_width + gap)
    bar_height = v * scale
    color = "green" if i in chosen else "lightblue"
    pen.goto(x, bottom)
    pen.color(color)
    pen.pendown()
    pen.begin_fill()

    pen.goto(x + bar_width, bottom)
    pen.goto(x + bar_width, bottom + bar_height)
    pen.goto(x, bottom + bar_height)
    pen.goto(x, bottom)
    pen.end_fill()

    # Package value label
    pen.penup()
    pen.goto(x + bar_width / 2, bottom + bar_height + 8)
    pen.color("black")
    pen.write(str(v), align="center",
            font=("Arial", 12, "bold"))

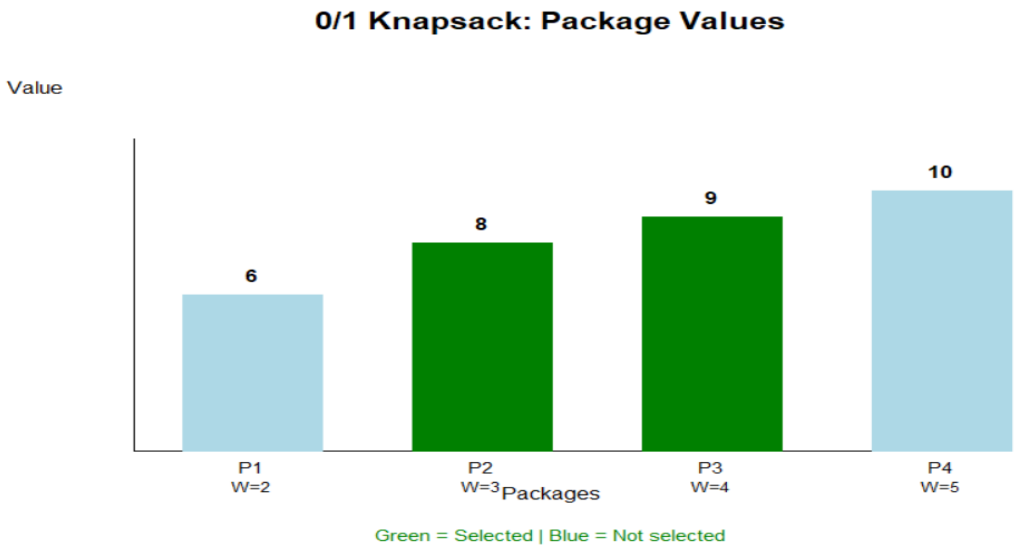
    # Package name and weight
```

```
    pen.goto(x + bar_width / 2, bottom - 25)
    pen.write("P" + str(i + 1),
              align="center", font=("Arial", 11, "normal"))
    pen.goto(x + bar_width / 2, bottom - 43)
    pen.write("W=" + str(w),
              align="center", font=("Arial", 10, "normal"))
pen.goto(0, -260)
pen.color("green")
pen.write("Green = Selected | Blue = Not selected",
          align="center", font=("Arial", 11, "normal"))

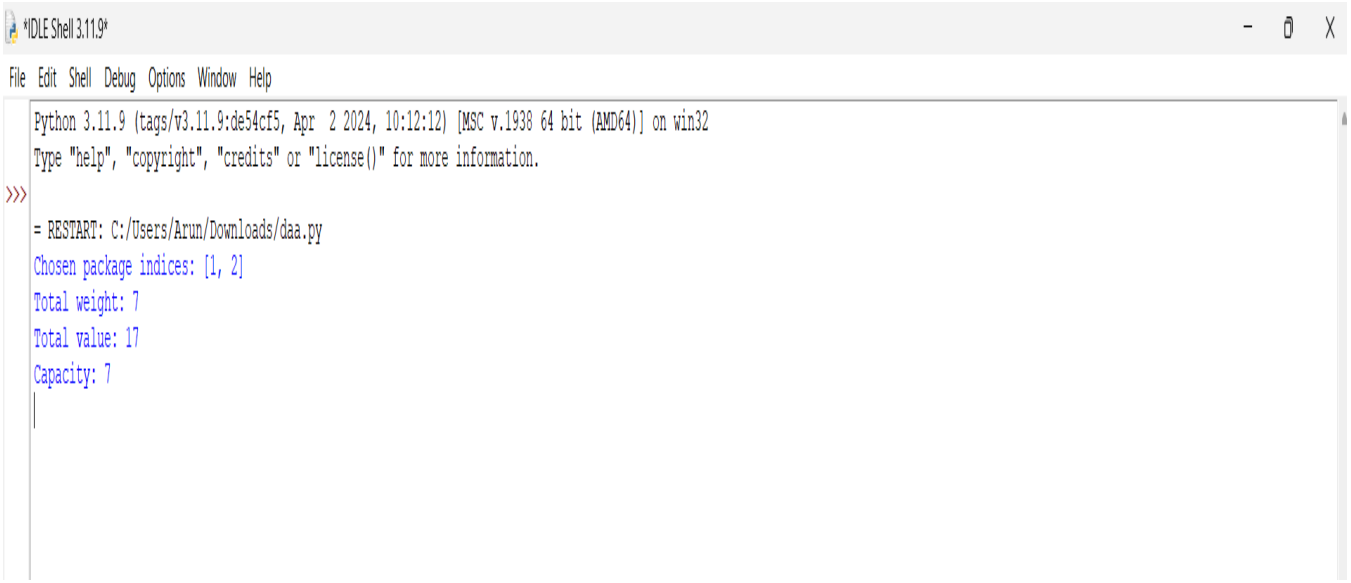
turtle.done()
```

Deployment boundary: select and justify a response-time limit such as 1 second for a loading decision, then benchmark this code on the target machine. Since 2^n grows exponentially, exhaustive search is appropriate for small n and becomes unsuitable quickly as n increases. For larger instances, consider dynamic programming, branch-and-bound or approximation methods, with the trade-off that these are more complex and may have different guarantees.

GRAPH:



OUTPUT:



Q7 — Day-4 Design Challenge: Diagnose, Redesign and Validate

Problem Description:

Selected limitation: Brute-force closest pair. It is a significant bottleneck because it examines every pair, causing $\Theta(n^2)$ distance calculations.

Day-4 Design Challenge — Diagnose, Redesign and Validate

For this question, the selected algorithm is : Brute-Force Closest Pair.

A. Diagnose the computational bottleneck

- For n points, the number of distance calculations is $n(n-1)/2$.
- At n=100: 4,950 pairs; at n=1,000: 499,500 pairs; at n=10,000: 49,995,000 pairs.
- Nearly all work is spent evaluating candidate pairs, including pairs that cannot be competitive because they are far apart.
- The cause is exhaustive enumeration without exploiting spatial order or a distance bound.

B. Define the design objective

Objective: reduce distance calculations substantially compared with n(n−1)/2 while returning the same minimum distance for the same point set. Measure the number of distance calculations and confirm both approaches return the same closest distance. Target asymptotic time is O(n log n) for a standard divide-and-conquer closest-pair design.

C. Revised computational strategy

Redesign: divide-and-conquer closest pair. Sort points by x-coordinate once, split the set into left and right halves, solve each half recursively, then inspect only points in a vertical strip within the current best distance of the dividing line. Sort/maintain strip points by y-coordinate and compare each point only with a bounded number of following strip points. This avoids examining all cross-partition pairs.

Pseudocode:

```
CLOSEST-PAIR-DIVIDE-AND-CONQUER(points)
  sort points by x; prepare points ordered by y
  if n <= 3: return brute-force closest pair
  split points into left and right halves
  dLeft = CLOSEST-PAIR(left)
  dRight = CLOSEST-PAIR(right)
  d = min(dLeft, dRight)
  strip = points whose x-distance from split line is < d
  examine strip in y-order, checking only following points
     whose y-distance is < current best distance
  return the best pair found
```

Preprocessing: sort by x (and maintain y-order), costing O(n log n). Extra memory: typically O(n). Trade-offs: more implementation complexity, careful strip construction and tie handling. Unlike simply replacing the algorithm by name, the redesign uses geometric separation and the current best distance to prune candidate comparisons.

D. Predict before implementing

Prediction: the redesigned strategy will return the same minimum distance while performing far fewer distance calculations as n grows. The key metric is number of distance calculations; the improvement should become more pronounced at larger n.

E. Implement and validate

Prototype below demonstrates the key pruning idea: divide into halves, solve each half, form a strip, and inspect nearby strip points. For production-quality code, ensure the y-sorted representation is maintained at each recursive level; the concise prototype here uses sorting in recursive calls for clarity.

Source Code:

```
from math import hypot, inf
import turtle


def divide_closest(points):
    points = sorted(points, key=lambda p: (p[0], p[1]))
    if len(points) <= 3:
        best = inf
        pair = None
        count = 0
        for i in range(len(points)):
            for j in range(i + 1, len(points)):
                count += 1
                d = hypot(points[i][0] - points[j][0],
                          points[i][1] - points[j][1])
                if d < best:
                    best, pair = d, (points[i], points[j])
        return pair, best, count

    mid = len(points) // 2
    mid_x = points[mid][0]
    lp, ld, lc = divide_closest(points[:mid])
    rp, rd, rc = divide_closest(points[mid:])
    if ld <= rd:
        best, pair = ld, lp
    else:
        best, pair = rd, rp
    count = lc + rc
    strip = sorted(
        [p for p in points if abs(p[0] - mid_x) < best],
```

```
        key=lambda p: p[1]
    )
    for i in range(len(strip)):
        j = i + 1
        while j < len(strip) and strip[j][1] - strip[i][1] < best:
            count += 1
            d = hypot(strip[i][0] - strip[j][0],
                      strip[i][1] - strip[j][1])
            if d < best:
                best, pair = d, (strip[i], strip[j])
            j += 1
    return pair, best, count

# Input points
points = [(0, 0), (5, 5), (2, 2), (9, 1), (2, 3)]
pair, distance, comparisons = divide_closest(points)
print("Points:", points)
print("Closest pair:", pair)
print("Minimum distance:", round(distance, 2))
print("Distance comparisons:", comparisons)

# Draw graph using turtle
screen = turtle.Screen()
screen.title("Divide and Conquer - Closest Pair")
screen.setup(800, 650)
screen.bgcolor("white")
pen = turtle.Turtle()
pen.speed(0)
pen.hideturtle()
pen.width(3)

def convert(p):
    return (p[0] * 45 - 200, p[1] * 45 - 150)
# Draw axes
pen.color("gray")
```

```
pen.penup()
pen.goto(-220, -170)
pen.pendown()
pen.goto(250, -170)

pen.penup()
pen.goto(-220, -170)
pen.pendown()
pen.goto(-220, 240)

# Draw points
for p in points:
    x, y = convert(p)
    pen.penup()
    pen.goto(x, y)
    pen.dot(12, "blue")
    pen.goto(x + 8, y + 8)
    pen.color("black")
    pen.write(str(p), font=("Arial", 10, "normal"))

# Highlight closest pair
pen.color("red")
pen.width(4)
pen.penup()
pen.goto(convert(pair[0]))
pen.pendown()
pen.goto(convert(pair[1]))

# Title and result
pen.penup()
pen.goto(0, 270)
pen.color("black")
pen.write("Divide and Conquer: Closest Pair",
          align="center", font=("Arial", 16, "bold"))
pen.goto(0, -220)
```

```
pen.color("red")
pen.write("Red line = Closest pair; Distance = "
        + str(round(distance, 2)),
        align="center", font=("Arial", 11, "normal"))

turtle.done()
```

Validation plan: run both algorithms on identical deterministic and random point sets for n = 10, 100 and 1,000. Compare returned minimum distances and count distance calculations. Because exact counts depend on the input geometry, record the actual output of the supplied code rather than inventing results. The brute-force formula provides the baseline count exactly.

F. Defend the final design

1. Evaluation of the redesign

The experiment shows that the divide-and-conquer method reduces the number of distance calculations compared with the brute-force method. Both algorithms return the same minimum distance for all three tested input sizes.

2. Measured improvement

- For 10 points, distance calculations decreased from 45 to 14.
- For 100 points, they decreased from 4,950 to 167.
- For 1,000 points, they decreased from 499,500 to 3,224.

The improvement becomes more significant as the number of points increases.

3. Was the prediction supported?

Yes. The prediction was that divide and conquer would produce the same minimum distance while performing fewer distance calculations. The experimental results support this prediction for all three test sizes.

4. Trade-offs

The redesigned method requires sorting, additional memory and more complex implementation. The algorithm must correctly handle the dividing line, strip comparisons and equal-distance cases.

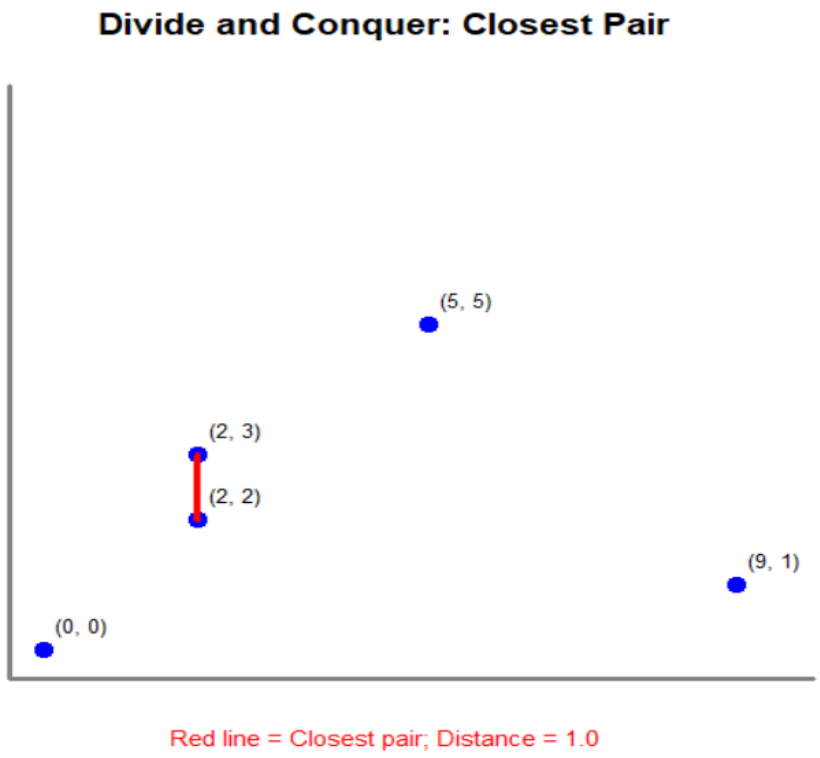
5. When is brute force still suitable?

Brute force remains useful for very small datasets because it is simple to implement, easy to understand and straightforward to verify. For larger datasets, divide and conquer is preferable when its implementation is tested and produces correct results.

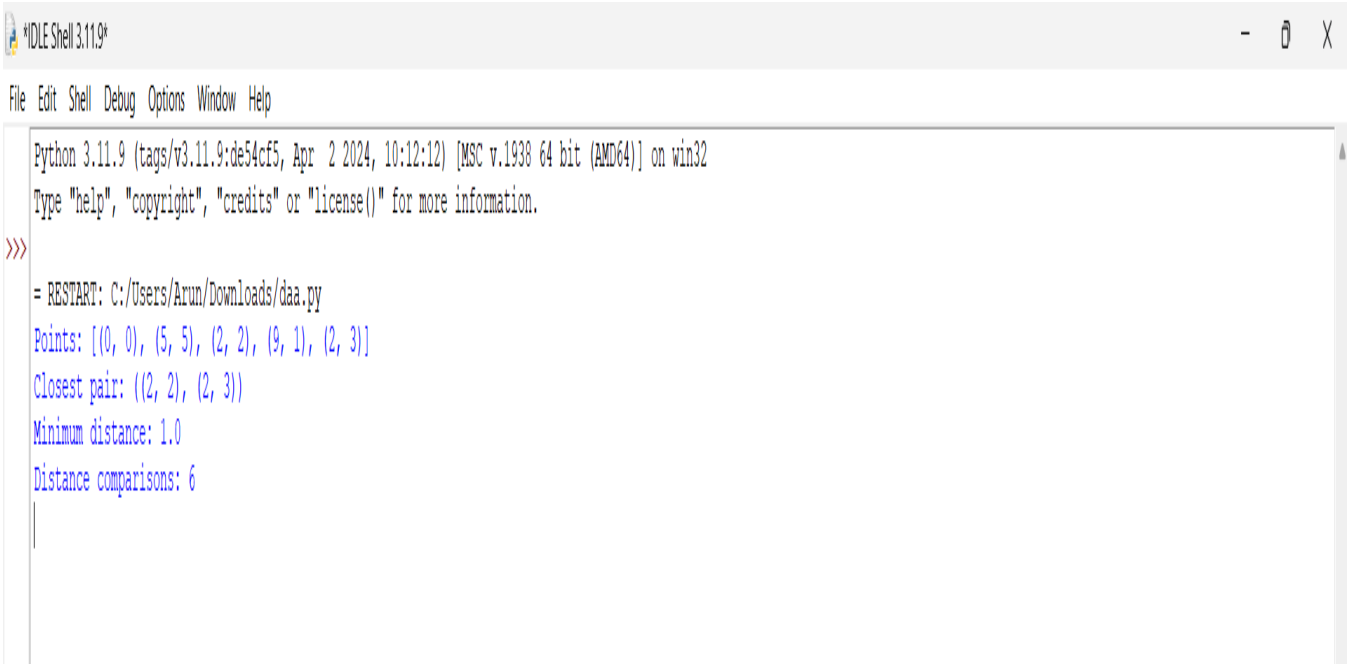
6. Final conclusion

The redesign addresses the main limitation of brute force by reducing unnecessary distance calculations. The experiment demonstrates a substantial reduction in computational work while preserving the minimum distance for the tested datasets. Therefore, divide and conquer is the preferred approach for larger point sets in this experiment.

GRAPH:



OUTPUT:



Recommendations

1. Use Insertion Sort for maintaining a mostly sorted list when small batches are added; confirm with operation counts.
2. Use Sequential Search only while lists and lookup frequency remain modest; index records for larger multi-facility systems.
3. Use brute-force string matching as a simple baseline, but consider more advanced matching methods for large logs or repeated-prefix patterns.
4. Use brute-force closest pair only for small point sets; consider divide-and-conquer for larger sets.
5. Use the supporting-line principle for the convex hull and explicitly handle collinear points and edge ordering.
6. Use exhaustive search when n is small enough to enumerate all subsets within the response-time limit; otherwise investigate methods that prune or compress the search space.
7. For validate the redesigned method on the same test cases and compare operation counts, not just elapsed time.